

아이디어에서 실행까지, AI와 함께하는 코딩

AI 기반 Vibe Coding · 입문 워크숍

작은 프로그램 하나를
직접 만들고 고쳐보며
코딩의 기본을 익힙니다.

9월 29일 · 16:00-17:00

경영기획본부 경영기획DX추진TF팀 오명철 과장

AI 에이전트에 요청해 테트리스 만들기

게임 만들기를 요청하고, 에이전트가 만든 파일과 실행된 게임을 확인합니다.

VS Code

파일 편집 보기 터미널

탐색기

TETRIS-WORKSHOP

index.html

tetris-workshop — Visual Studio Code

index.html

tetris-workshop / index.html

1 <!-- 여기에 작성하세요 -->

Codex

로컬 작업

개발 과정 재현

테트리스 첫 버전

먼저 돌아가는 게임을 만듭니다. 자동 플레이는 처음부터 빠르게 실행합니다.

테트리스 첫 버전 만들기

tetris-workshop/

작업 폴더 사용

main

요청 대기

VS Code + Codex 구성 재현

JavaScript

UTF-8

첫 버전은 투박해도 됩니다. 자동으로 움직이는 결과를 보고 다음 요청을 정합니다.

추가 요청으로 테트리스 기능 개선하기

화면 디자인, 점수판, 착지 효과를 차례로 요청하고 게임이 어떻게 달라지는지 비교합니다.

VS Code

파일 편집 보기 터미널

탐색기

TETRIS-WORKSHOP

index.html A

style.css A

tetris.js A

JS tetris.js

tetris-workshop / tetris.js

1 // 자동 플레이는 처음부터

2 const AUTO_PLAY = true;

3 const DROP_MS = 40;

4

5 // 요청에 따라 바뀌는 설정

6 const options = {

7 color: false,

8 nextPreview: false,

9 largeScore: false,

10 ghostPiece: false,

11 clearEffect: false

12 };

13

14

tetris-workshop — Visual Studio Code

브라우저 미리보기

localhost:5500/index.html

TETRIS

AUTO · 빠른 자동 플레이

SCORE
0

LINES
0

실행 대기

첫 실행을 기다립니다

Codex에 제작을 요청해 보세요.

자동 ON 일시정지 새 게임

터미널 문제 O 출력

PowerShell

PS C:\work\tetris-workshop>

개발 과정 재현

로컬 작업

Codex

같은 게임을 하나씩 개선합니다

색과 화면, 점수판, 착지 효과를 차례로 요청해 보세요. 게임판은 이어집니다.

① 블록 색과 화면 디자인

② 큰 점수판과 다음 블록

③ 착지 위치와 줄 삭제 효과

@tetris.js · @style.css

작업 폴더 사용

main

✓ 요청 대기

VS Code + Codex 구성 재현

JavaScript

UTF-8

AI가 코딩할 때, 사람은 무엇을 할까요?

사람은 만들 목적을 정하고, AI가 구현한 결과를 보며 다음에 고칠 부분을 판단합니다.

01

목적 정하기

“쉽게 즐길 수 있는 게임”
무엇을 만들지 결정합니다.

02

AI의 구현

허용된 도구로 파일을 만들고
수정하며 실행을 돕습니다.

03

결과 판단하기

“빠르다”, “잘 안 보인다”
사용 경험과 정답을 확인합니다.

AI가 개발을 맡으면, 사람은 그 개발을 이끄는 리더가 됩니다.

무엇을 왜 만들지 설명하고, 우선순위를 정하고,
결과가 목적에 맞는지 확인하며 다음 개선을 이끍니다.

이제 같은 기능을 두 방식으로 만들어 보겠습니다.



일반 챗봇과 코딩 에이전트, 무엇이 다를까?

같은 기능을 요청해 봅니다. 일반 챗봇의 코드는 사람이 옮겨 적용하고, 코딩 에이전트는 파일을 직접 수정하고 실행합니다.

미리보기·착지 가이드·줄 삭제 효과를 게임에 추가하기

1. 챗봇 · 사람이 코드 조각을 연결

요청 → 세 파일의 코드 답변 → 복사 → 화면과 스타일 적용 → 게임 로직의 위치 확인·연결 → 실행 확인

index.html → 다음 블록을 표시할 공간
style.css → 미리보기와 효과의 스타일
tetris.js → 착지 계산 · 다음 블록 그리기 · 줄 삭제 효과

함수를 붙여 넣는 것만으로는 부족합니다. 게임이 그 함수를 실행하는 위치까지 사람이 연결해야 합니다.

2. 코딩 에이전트 · 파일과 실행 흐름을 함께 수정

같은 요청 → 세 파일과 관련 함수 확인 → 파일 수정 및 실행 위치 연결 → 게임에서 결과 확인
실행 결과: 다음 블록이 보이고, 실제 충돌 위치에 착지 윤곽이 표시되며, 줄 삭제에 빛과 점수 효과가 붙습니다. 사람은 원하는 기능인지, 동작과 표현이 적절한지 판단합니다.

만든 테트리스, 무엇을 고쳐 달라고 할까요?

앞에서 만든 게임을 보며, 화면에 필요한 요소·보이는 방식·게임 동작을 나눠 요청합니다.



화면 구성 · 다음 블록이 안 보일 때

“게임판 옆에 다음 블록을 보여주는 영역을 추가해줘.”

디자인 · 점수가 잘 안 읽힐 때

“점수 글자를 키우고, 어두운 배경에서도 잘 보이는 밝은 색으로 바꿔줘.”

동작 · 다시 시작해도 점수가 남을 때

“새 게임을 시작하면 이전 점수가 남지 않도록 0점으로 초기화해줘.”

이 요청들은 어떤 파일에 반영될까요? 이제 AI가 만든 프로젝트 폴더를 열어봅니다.

AI가 만든 파일은 어디에 있고, 무슨 역할일까요?

프로젝트 폴더에서 화면·디자인·동작을 담은 파일과 작업 지침·실행 안내를 살펴봅니다.

tetris-workshop/

 index.html

style.css

 game.js

 AGENTS.md

 README.md

index.html · 화면 구조

`<h1>테트리스</h1>`

`<canvas id="board"></canvas>`

`<p>점수 0</p>`

화면에 제목, 게임판, 점수 표시가 있다는 구조를 표현합니다.

프로젝트 폴더는 코드와 자료를 함께 두는 작업 공간입니다.

처음에는 여러 역할을 HTML 파일 하나에 담을 수도 있습니다.

각 파일이 맡는 구조, 모양, 동작을 화면에서 살펴봅니다.

HTML·CSS·JavaScript는 각각 무엇을 할까요?

같은 화면에 구조, 디자인, 클릭 동작을 하나씩 더하며 세 가지 역할을 비교합니다.

5 HTML

+ 3 CSS

+ Js JavaScript

오늘의 작업

첫 프로그램 확인

만든 결과를 실행해 보고 확인합니다.

상태 **확인 전**

확인 완료

JavaScript가 버튼의 입력을 받아 상태를 바꿉니다.

각 역할을 담은 파일은 어떻게 실행해서 확인할까요?

HTML

제목, 입력란, 버튼 등 화면의 구성 요소

CSS

글자 크기, 색, 간격과 배치 등 화면의 표현 방식

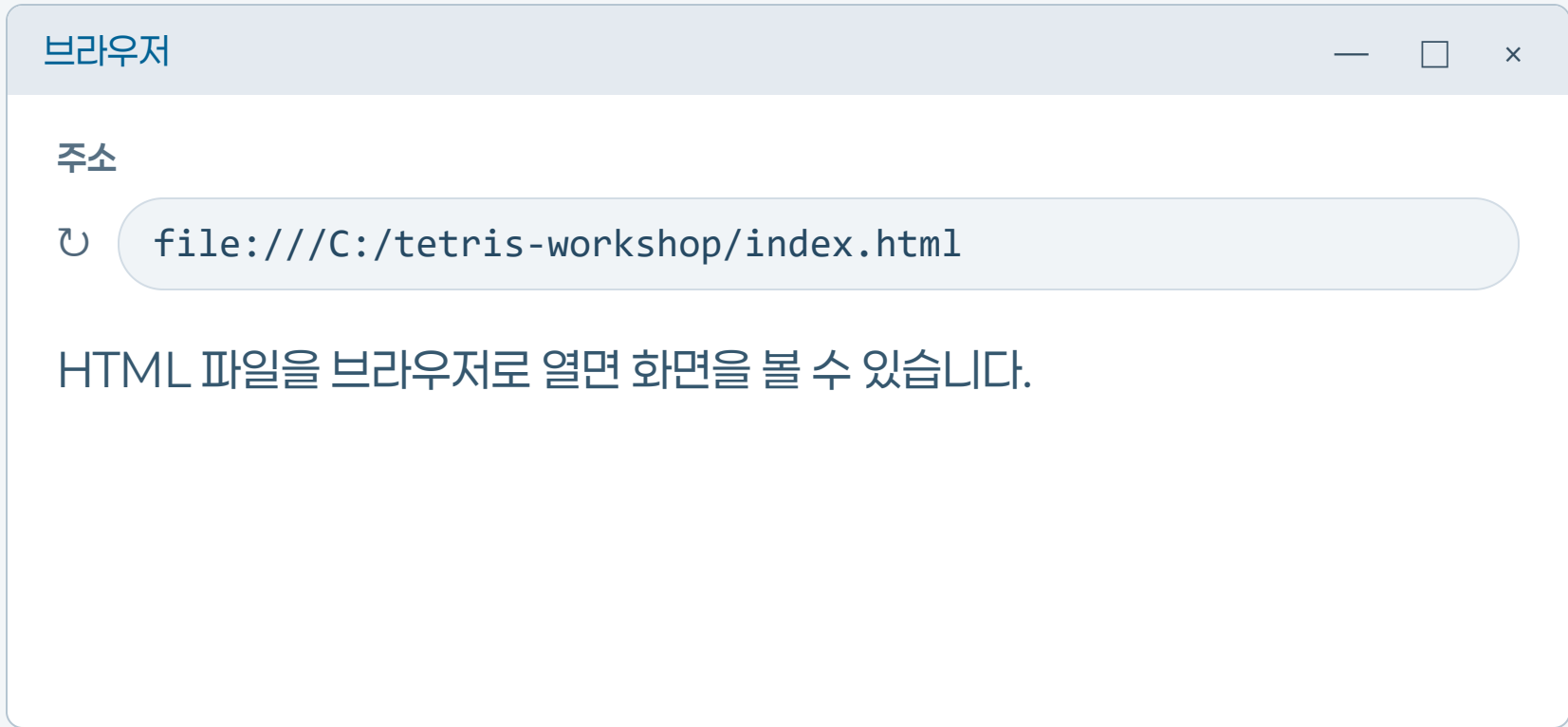
JavaScript

클릭·입력에 따른 화면과 데이터의 변화

HTML과 Python 파일은 어떻게 실행할까요?

HTML 파일 열기, Python 실행, Python 서버의 주소로 HTML 화면 열기를 비교합니다.

- HTML 게임
- Python 프로그램
- HTML+Python 혼합



실제로는 개발 플랫폼에 따라 실행 방법이 달라질 수 있습니다.

브라우저 화면과 Python 서버는 각각 어떤 역할을 맡을까요?

터미널

명령을 입력하고 결과를 보는 창입니다. 에이전트도 이 창을 통해 개발도구를 실행할 수 있습니다.

Python

Python으로 쓴 프로그램을 실행하려면 Python 실행 환경이 필요합니다.

잘 모르겠으면...? 그냥 AI Agent에게,

“이 프로그램 실행시켜줘” 라고 하세요 🤖

프론트엔드·백엔드·데이터베이스의 역할

사용자가 보는 화면, 서버의 요청 처리, 데이터 저장이 각각 어떤 일을 맡는지 살펴봅니다.



프론트엔드

사용자가 보고 조작하는 부분



백엔드

서버에서 요청을 처리하는 부분



데이터베이스

데이터를 저장하고 찾아 쓰는 부분

앞에서 본 작은 게임은 브라우저 안에서 실행됩니다.

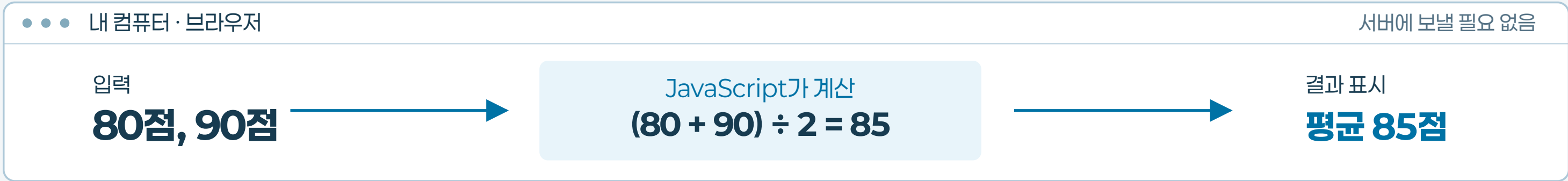
별도의 서버와 데이터베이스가 모든 프로그램에 필요한 것은 아닙니다.

서버가 있는 경우에는 요청과 결과가 어떻게 오갈까요?

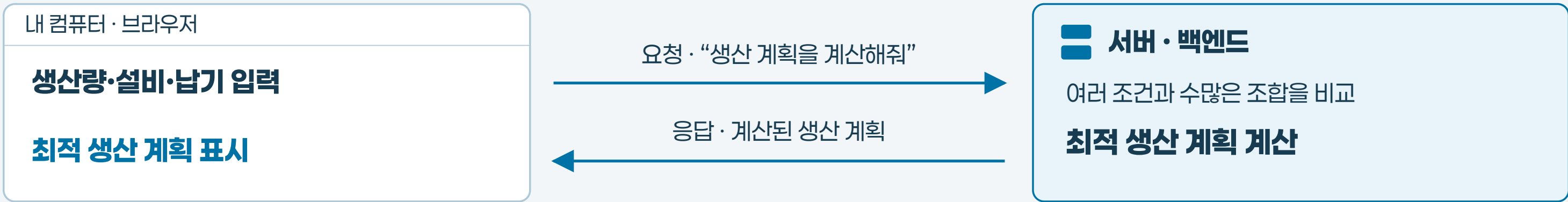
간단한 계산은 JS로, 복잡한 계산은 백엔드로

평균 계산은 브라우저에서 바로, 조건이 많은 생산 계획 계산은 서버에 맡기는 예시입니다.

① 간단한 계산 · JavaScript가 바로 처리



② 복잡한 계산 · 백엔드에 요청



API는 이 요청과 응답을 주고받는 약속입니다.
이 예시에서는 계산량이 큰 작업을 백엔드에 맡기고, 브라우저는 받은 결과를 보여줍니다.

처리 위치는 계산량과 필요한 데이터에 따라 정합니다. 백엔드에서도 JavaScript를 사용할 수 있습니다.

좋은 화면을 보는 안목이란?

화면에서 무엇이 불편한지 알아보고, 어떻게 바꾸면 쓰기 편해질지 판단하는 능력입니다.

- ☐ 읽기 편한 간격
- ☐ 중요한 정보를 먼저
- ☐ 분명한 버튼 이름
- ☐ 절제된 모서리 라운드

무엇이 불편한가요?
하나씩 바꾸며 비교해 보세요.

후보 검토

OK

마지막 변경 09.29

검토할 후보 3건

기준 점수 80점 · 2건 충족

후보 A	92 충족
후보 B	85 충족
후보 C	74 미달

이 판단이 있어야 시에게 “더 예쁘게”를 넘어, 어디를 왜 바꿀지 말할 수 있습니다.

막연한 “고쳐줘”가 아닌 구체적인 수정 요청으로

어디가 불편한지 짚고, 원하는 결과를 말하면 AI가 수정할 기준이 생깁니다.

눈으로 확인한 문제	AI에게 이렇게 요청하기	수정 후 확인할 모습 · 예시
01 화면 검토할 후보 3건 OK 버튼을 누르면 무엇이 시작될까?	“더 보기 좋게 해줘.” 버튼 이름이 ‘OK’라서 역할을 모르겠어. ‘검토 시작’으로 바꿔줘.	검토할 후보 3건 검토 시작 다음 행동이 버튼 이름에 드러납니다.
02 판단 기준 80점 → 미달 80점 이상이면 통과인데?	“계산이 이상해. 고쳐줘.” 80점 이상이면 충족이야. 80점도 ‘충족’으로 표시해줘.	79점 미달 80점 충족 81점 충족 기준 바로 아래와 위도 함께 확인합니다.
03 빠진 입력 점수 입력 → 미달 점수를 안 넣었는데 판정이 나왔네?	“오류 안 나게 해줘.” 빈칸은 0점이 아니야. 판정 대신 점수를 넣어 달라고 알려줘.	점수 입력 점수를 입력해 주세요. 입력 누락과 낮은 점수를 구분합니다.

요청의 핵심은 두 가지입니다. “지금 어떤 문제가 있는지” + “어떻게 되어야 하는지”.

AGENTS.md로 지침을, Git으로 되돌릴 지점을

반복해서 시키는 일은 지침으로 남기고, 중요한 작업 상태는 저장해 안심하고 수정합니다.

AGENTS.md · 항상 적용할 지침

AI 에이전트가 작업을 시작하기 전에 읽고 따르는 지침 파일

사용자가 반복해서 시키는 일을 적어 두면,
매번 말하지 않아도 작업마다 적용합니다.

반복해서 하던 요청

“원본은 보존하고, 결과는 output 폴더에 저장해줘.”

↓ AGENTS.md에 한 번 기록

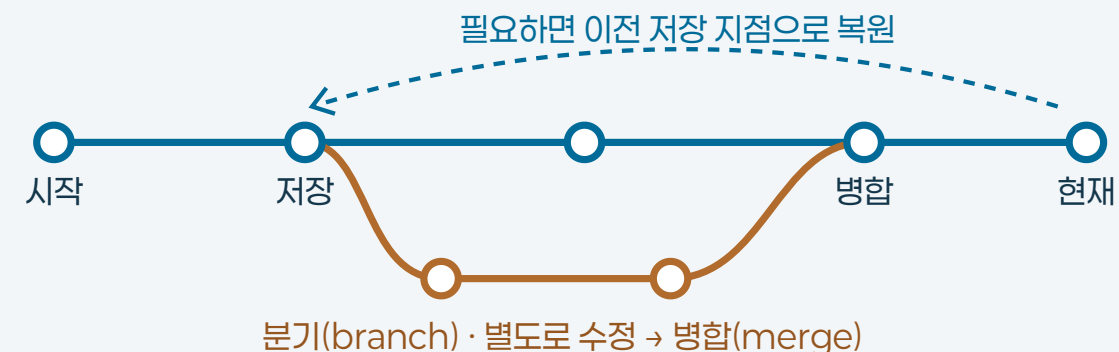
이후 작업에서도 같은 규칙을 따르도록

단, Claude의 경우 AGENTS.md가 아닌 CLAUDE.md를 기본으로 읽습니다.

AGENTS.md는 앞으로 지킬 지침, Git은 돌아갈 수 있는 작업 기록입니다.

Git · 저장하고, 되돌리고, 함께 작업하기

수정 전 상태를 남겨 두고, 문제가 생기면 이전 상태로 복원



전체를 매번 새로 복사하지 않습니다.

바뀌지 않은 파일은 재사용하고, 바뀐 코드 부분은 비교해 볼 수 있습니다.

저장·복원 커밋(●)으로 돌아갈 지점을 남깁니다.

원격 저장소에도 올려 두면 별도 백업이 됩니다.

여러 명의 협업에 필수 각자 작업을 나눠 진행하고,
변경 이력을 확인하며 하나로 합칩니다.

사내 LLM API로 챗봇을 만들어봅니다

API 호출 샘플을 바탕으로 질문을 입력하고 답변을 받는 챗봇을 함께 만듭니다.

오늘 만들 프로그램

질문을 보내면 답변이 오는 챗봇

“오른쪽 API 샘플을 활용해서
대화형 챗봇을 만들어줘.
질문 입력창과 전송 버튼,
대화 내역이 보이게 해줘.”

01 요청 샘플 코드와 원하는 화면 전달

02 확인 질문 전송 → 답변 표시 → 후속 질문

03 수정 응답 대기 표시와 오류 안내 추가

키는 Python 실행 환경의 PGPT_API_KEY에서 읽습니다. 대화
내역은 messages에 이어 보내도록 요청합니다.

사내 LLM API 호출 샘플

Python · requests

```
import os
import requests

url = "http://pgpt.posco.com/s01a01-gpt/gptApi/personalApi"
api_key = os.getenv("PGPT_API_KEY")

headers = {
    "accept": "*/*",
    "Authorization": f"Bearer {api_key}",
    "Content-Type": "application/json"
}
data = {
    "model": "gpt-5.6-luna",
    "messages": [
        {"role": "system", "content": "시스템 지시"},
        {"role": "user", "content": "테스트"}
    ]
}
response = requests.post(url, headers=headers, json=data)
print('\n', response.text, '\n')
```

API 키는 환경변수나 .env에 보관하세요

API 키는 내 계정으로 AI를 쓰게 해 주는 비밀 열쇠입니다. 코드에 직접 적지 않고 따로 관리합니다.

01 키는 코드 밖에 저장

환경변수는 실행할 때 전달하는 설정값,
.env는 그 값을 적어 두는 파일입니다.

.env 파일 예시

API_KEY=여기에_발급받은_키

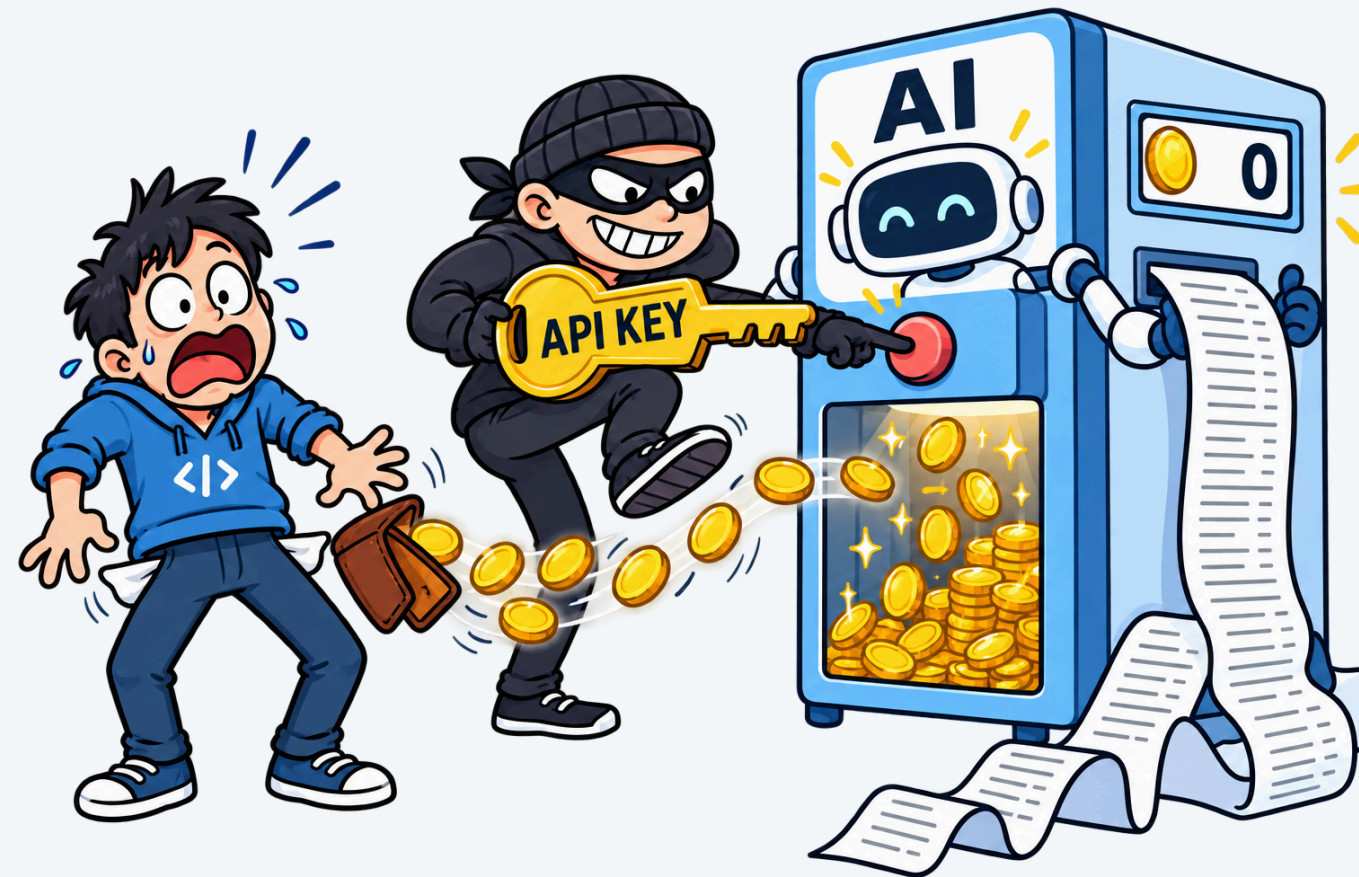
설명용 예시입니다. 실제 키를 화면에 공유하지 마세요.

02 프로그램은 저장한 값을 읽어 사용

AI에게 “키를 환경변수에서 읽도록 해줘.
.env를 쓸 경우 불러오는 설정도 해줘.”

03 공유 전 확인도 AI에게 맡기기

“공유할 파일에 API 키가 들어 있지 않은지
확인하고, 필요한 설정도 해줘.”



“AI는 도둑이 쓰고, 요금은 내가?!”

키가 유출되면 충전 잔액이 바닥날 수 있어요.
결제 설정에 따라 추가 요금도 생길 수 있습니다.
유출됐다면 즉시 키를 폐기하고 새로 발급받으세요.

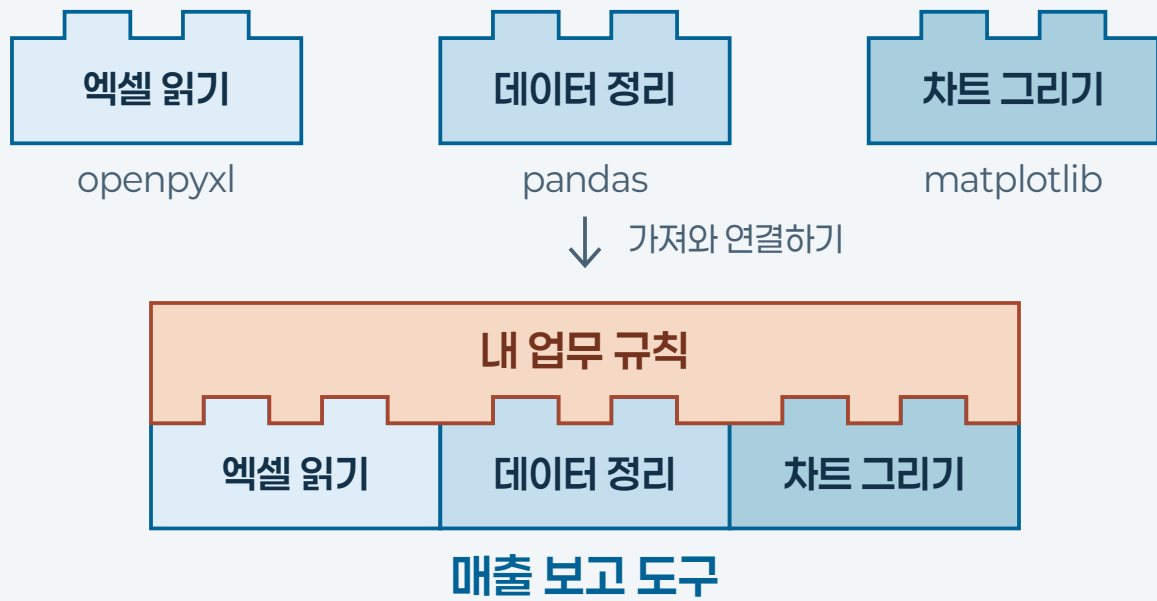
기억할 것은 하나: API 키는 환경변수나 .env에. 설정과 확인은 AI에게 요청하세요.

코딩에는 어떤 재료와 준비가 필요할까요?

모든 기능을 처음부터 만들지 않습니다. 공개된 오픈소스를 가져와, 내 목적에 맞게 연결하고 필요한 부분을 만듭니다.

이미 있는 기능을 블록처럼 조립합니다

엑셀 읽기·데이터 정리·차트 그리기를 가져와
우리 업무의 기준과 흐름을 코드로 연결합니다.



라이브러리 가져다 쓰는 기능 모음

패키지 설치·배포할 수 있게 묶은 단위

오픈소스는 소스 코드가 공개되어, 라이선스 조건에 따라
사용·수정·배포할 수 있는 소프트웨어입니다.

Python 프로젝트라면 이렇게 준비합니다

01 실행할 Python + 프로젝트 폴더

코드를 실행할 도구와 작업 파일을 둘 곳이 필요합니다.

02 프로젝트 전용 가상환경

패키지를 따로 설치해 프로젝트 간 버전 충돌을 줄입니다.
컴퓨터 전체를 가상화하는 가상 머신과는 다릅니다.

03 필요한 패키지 설치

pip는 설치 도구, uv는 환경·패키지·실행 관리 도구입니다.

pip로 설치

```
python -m pip install pandas
```

가상환경의 Python으로 실행 · 흔히 pip install로 표현

uv로 준비하고 실행

```
uv init     프로젝트 시작
```

```
uv add pandas     패키지 추가
```

```
uv run main.py     코드 실행
```

uv 설치 후 프로젝트 폴더에서 · .venv 환경 자동 관리

설치 후 코드의 `import pandas`로 기능을 불러 씁니다.



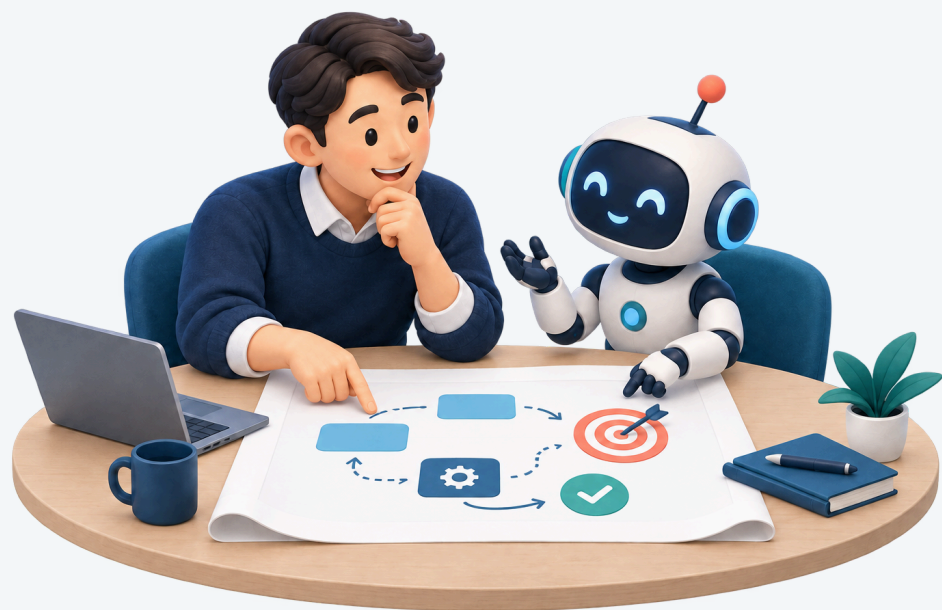
시에게 “필요한 라이브러리와 이유를 설명하고, 이 프로젝트의 가상환경에 설치한 뒤 실행해줘.”

복잡한 작업은, AI와 생각을 맞춘 뒤 시작하기

대화로 목표와 방법을 구체화하고, 서로 이해한 내용이 맞는지 확인한 뒤 코딩을 시작합니다.

코딩 전에, 서로 같은 그림부터

원하는 결과와 제약을 설명하고,
AI의 질문과 제안으로 빠진 조건을 채웁니다.



01 원하는 결과를 말하고, 질문받기

나 “여러 엑셀 파일을 모아 월간 보고서를 만들고 싶어.
바로 코딩하지 말고, 먼저 필요한 걸 질문해 줘.”

02 방법을 비교하고, 조건 구체화하기

AI “파일마다 열 이름이 같나요? 중복은 어떻게 처리하죠?
먼저 샘플로 통합 방식을 확인하면 어떨까요?”

질문에 답하고 제안을 조정하며, 작업 방법과 예외 처리를 정합니다.

03 이해한 내용을 맞춘 뒤, 코딩 시작하기

나 “중복은 한 건만 남기자. 만들 결과와 작업 순서,
확인할 기준을 정리해 줘. 서로 맞는지 보고 시작하자.”

여러분은 어떤 부분이 궁금하신가요?



용어 자세히 보기

Vibe Coding

원하는 기능을 자연어로 설명하고, AI가 만든 코드를 실행하고 수정하며 프로그램을 만드는 접근입니다.

이 교육에서는 요청, 실행, 결과 확인, 수정의 흐름으로 경험합니다. AI가 코드를 작성하더라도 목적과 업무 기준을 설명하고 결과를 판단하는 일이 필요합니다.

예를 들어 테트리스를 만든 뒤, 점수판 추가와 블록 색 변경을 요청하고 게임에 반영됐는지 확인합니다.

브라우저

웹 페이지를 열고 사용하는 프로그램입니다.

Edge, Chrome 등이 해당합니다. HTML과 CSS로 화면을 표시하고 JavaScript를 실행합니다. 브라우저에서 할 수 있는 일과 서버에서 하는 일은 구분해서 봅니다.

오늘의 HTML 파일은 브라우저에서 직접 열어 사용할 수 있고, 외부 서비스 연결 없이 준비된 체험이 동작합니다.

서버

다른 프로그램의 요청을 받아 기능이나 데이터를 제공하는 프로그램 또는 그 실행 환경입니다.

브라우저가 요청하는 계산, 저장, 인증 같은 일을 맡을 수 있습니다. 서버 쪽 처리 부분을 백엔드라고 부릅니다. 모든 작은 도구에 별도 서버가 필요한 것은 아닙니다.

여러 사람이 같은 자료를 조회하거나, 비밀 키를 사용해 AI 서비스에 요청할 때 서버 쪽 처리를 둘 수 있습니다.

Node.js

JavaScript를 브라우저 밖에서 실행하는 환경입니다.

서버 프로그램이나 일부 개발도구가 이를 사용합니다. JavaScript를 사용하는 모든 화면에 별도로 설치해야 하는 것은 아닙니다.

오늘처럼 브라우저에서 여는 단일 HTML 파일에는 Node.js 설치가 필요하지 않습니다. 만들 프로그램에 맞춰 필요한지 확인합니다.

패키지

프로그램 기능이나 도구를 설치하고 배포할 수 있게 묶은 단위입니다.

라이브러리나 실행 프로그램이 담길 수 있습니다. 패키지 관리자는 이런 묶음을 설치하고 버전을 관리하는 도구입니다.

Python의 pip, JavaScript 생태계의 npm 등이 있습니다. 특정 패키지가 왜 필요한지, 어떻게 설치하는지는 에이전트에게 물어볼 수 있습니다.

Markdown

제목과 목록 같은 문서 구조를 간단한 문자 기호로 표현하는 형식입니다.

파일 확장자는 보통 .md입니다. 코드를 실행하는 프로그램이 아니라 사람이 읽을 설명과 규칙을 적는 데 자주 사용합니다.

README.md에는 실행 안내를, AGENTS.md에는 지원하는 에이전트가 참고할 작업 기준을 기록할 수 있습니다.

코딩 에이전트

파일과 도구를 사용해 코딩 작업을 수행하는 AI 시스템입니다.

일반적인 코드 답변을 넘어, 연결된 작업 공간에서 파일을 읽고 수정하고 명령을 실행할 수 있습니다. 가능한 행동은 제품, 연결된 도구, 권한에 따라 달라집니다.

게임의 점수 표시를 수정하고 실행 결과를 확인하도록 요청할 수 있습니다. 완료 메시지와 실제 결과는 함께 확인합니다.

GUI

Graphical User Interface. 화면의 버튼, 입력란, 목록 등으로 조작하는 사용자 인터페이스입니다.

정보가 어떤 순서로 보이는지, 글자가 읽히는지, 다음 행동이 분명한지를 함께 봅니다. 보기 좋은 장식만 뜻하지 않습니다.

“예쁘게”보다 “검토 건수를 먼저 보여주고 주요 버튼의 이름을 구체적으로 바꿔줘”라고 요청할 수 있습니다.

프로젝트 폴더

하나의 프로그램에 필요한 파일과 자료를 모아 둔 폴더입니다.

코드, 참고 자료, 설정, 실행 안내를 함께 관리합니다. 에이전트가 어느 폴더에서 작업하는지 확인해야 다른 작업과 섞이지 않습니다.

tetris-workshop 폴더에 index.html과 실행 안내를 두고, 그 폴더에서 변경을 이어갑니다.

터미널

문자 명령을 입력하고 프로그램의 출력을 보는 창입니다.

PowerShell 같은 셸이 명령을 해석합니다. CLI는 버튼 대신 명령어로 도구를 사용하는 방식을 뜻합니다. 터미널, 셸, CLI는 연결된 개념이지만 같은 뜻은 아닙니다.

python app.py에서 python은 실행할 도구, app.py는 실행할 파일입니다. 파일 위치와 설치 상태가 맞아야 실행됩니다.

Python

데이터 처리나 서버 프로그램 등에 사용하는 프로그래밍 언어입니다.

Python 코드를 실행하는 환경을 설치해야 해당 코드를 동작시킬 수 있습니다. 웹 화면을 구성하는 HTML과 역할이 다릅니다.

점수를 읽어 계산하는 app.py를 Python으로 실행할 수 있습니다. 모든 작은 HTML 도구에 Python이 필요한 것은 아닙니다.

HTML

웹 화면의 구조와 내용에 의미를 부여하는 마크업 언어입니다.

제목, 문단, 입력란, 버튼 같은 요소를 표현합니다. 그 자체가 AI 기능이거나 서버를 뜻하지는 않습니다.

버튼이 있다는 구조는 HTML로, 버튼의 색과 간격은 CSS로, 누른 뒤의 변화는 JavaScript로 구성할 수 있습니다.

CSS

화면의 색, 크기, 간격, 배치 같은 표현을 정하는 스타일 언어입니다.

같은 HTML이어도 CSS에 따라 읽는 순서와 사용 경험이 달라집니다. 화면 크기에 맞는 배치도 다릅니다.

점수를 크게 표시하고 주된 버튼을 눈에 띄게 배치합니다. 색을 바꾼다고 잘못된 계산이 고쳐지는 것은 아닙니다.

JavaScript

웹 화면의 입력과 상태 변화 등을 구현하는 프로그래밍 언어입니다.

브라우저에서 버튼 클릭에 반응하고 데이터를 계산할 수 있습니다. 브라우저 밖의 실행 환경에서도 사용할 수 있습니다.

오늘의 순위 도구는 브라우저 안에서 입력을 읽고 정렬합니다. 외부 시나 별도 서버를 호출하지 않습니다.

프론트엔드

사용자가 직접 보고 조작하는 부분입니다.

웹 프로그램에서는 브라우저에 표시되는 화면과 그 동작이 대표적입니다. 화면 구성뿐 아니라 입력 검사나 일부 계산도 할 수 있습니다.

이름과 점수를 입력하는 칸, 정렬 결과, 오류 안내가 해당합니다. 모든 처리 로직이 반드시 백엔드에 있어야 하는 것은 아닙니다.

백엔드

서버 쪽에서 요청을 받아 처리하는 부분입니다.

데이터 확인, 공통 업무 규칙, 권한 검사, 저장, 다른 서비스 호출 등을 맡을 수 있습니다. 브라우저만으로 충분한 도구에는 별도 백엔드가 없을 수도 있습니다.

여러 사용자가 공유하는 자료를 저장하거나, 서버에서 인증 정보를 사용해 시 서비스에 요청할 때 필요할 수 있습니다.

데이터베이스

데이터를 체계적으로 저장하고 조회하는 저장소입니다.

화면을 닫은 뒤에도 자료를 보관하거나 여러 조건으로 찾아 쓰는 데 활용합니다. 어떤 자료를 어떻게 보관할지에 따라 구조가 달라집니다.

현재 순위 도구는 입력을 메모리에서만 처리합니다. 계속 보관하려면 파일 저장이나 데이터베이스 등 저장 방법을 별도로 설계해야 합니다.

API

Application Programming Interface. 프로그램끼리 정해진 방법으로 기능을 요청하고 결과를 받는 점점입니다.

웹 API에서는 주소, 요청 형식, 인증 방식과 응답 형식 등을 약속합니다. API는 AI에만 있는 것이 아닙니다.

브라우저가 서버에 점수 계산을 요청하거나, 백엔드가 AI 서비스에 문장 분류를 요청할 수 있습니다. 실패 응답도 처리해야 합니다.

AGENTS.md

AI 에이전트가 작업을 시작하기 전에 읽고 따르는 프로젝트 지침 파일입니다.

사용자가 반복해서 요청하는 방식과 꼭 지킬 규칙을 적어 두면 매번 다시 설명하지 않아도 작업마다 적용할 수 있습니다. 단, Claude의 경우 AGENTS.md가 아닌 CLAUDE.md를 기본으로 읽습니다.

AGENTS.md를 지원하는 에이전트에서 사용하며, 읽는 위치와 적용 범위는 도구에 따라 다릅니다.

Git

작업 상태를 기록해 이전 상태로 되돌리고, 여러 사람의 변경을 합칠 수 있는 버전 관리 도구입니다.

커밋은 프로젝트 전체 상태를 가리키는 저장 지점입니다. 매번 전체 파일을 새로 복사하지 않고, 바뀌지 않은 파일은 기존 내용을 재사용합니다. 변경된 코드 부분은 차이 비교로 확인할 수 있습니다. 브랜치는 작업 흐름을 나누고, 머지는 나눈 작업을 합칩니다.

수정 전에 커밋을 남기면 문제가 생겼을 때 복원할 수 있습니다. GitHub 같은 원격 저장소에 올려 두면 별도 백업과 팀 협업에도 활용할 수 있습니다.

API 키

서비스를 사용하는 주체나 권한을 확인하는 인증 정보입니다.

노출되면 다른 사람이 권한이나 사용량을 악용할 수 있습니다. 브라우저에 전달한 코드와 값은 사용자에게 보일 수 있으므로, 비밀 키는 서버 측에서 관리합니다.

발표자료·공유 파일·브라우저 화면에 실제 키를 넣지 않습니다. 이 자료에는 실제 키 입력란이나 서비스 연결이 없습니다.

환경변수

실행 환경에서 프로그램으로 전달하는 설정값입니다.

API_KEY라는 이름에 발급받은 키를 설정하면 프로그램은 이름으로 그 값을 읽을 수 있습니다. 코드에 실제 키를 적지 않아도 됩니다.

환경변수나 .env가 자동으로 암호화해 주지는 않습니다. 키를 로그에 출력하거나 브라우저에 전달하지 않습니다.

.env

API_KEY=값처럼 실행 설정을 적어 두는 파일입니다.

프로젝트에서 .env를 읽도록 설정해야 값이 프로그램에 전달됩니다. 파일만 만들었다고 모든 프로그램이 자동으로 읽는 것은 아닙니다.

실제 키가 든 .env는 공유하거나 GitHub에 올리지 않습니다. 웹 서비스에서는 서버 측에서 불러오며, 브라우저에 포함되는 공개 설정으로 사용하지 않습니다.

.gitignore

Git에 새로 기록하지 않을 파일을 지정하는 목록입니다.

.gitignore 파일에 .env를 한 줄로 적으면 Git이 아직 추적하지 않는 .env 파일을 제외할 수 있습니다.

이미 Git에 기록한 파일에는 소급 적용되지 않습니다. 키가 올라갔다면 파일을 지우는 것만으로는 부족하므로 키를 즉시 폐기하고 재발급합니다.

라이브러리

다른 프로그램에서 가져다 쓸 수 있도록 만든 기능 모음입니다.

패키지는 설치·배포 가능한 묶음이고 라이브러리를 포함할 수 있습니다. 패키지 관리자는 이런 묶음의 설치와 버전을 관리합니다.

표 읽기, 차트 그리기 같은 기능을 가져다 쓸 수 있습니다. 무엇이 필요한지는 만드는 프로그램에 따라 달라집니다.

가상환경

Python 프로젝트마다 패키지 설치 환경을 분리하는 공간입니다.

서로 다른 프로젝트에서 필요한 패키지 버전이 충돌하는 일을 줄입니다. 가상 머신이나 별도의 컴퓨터를 만드는 것은 아닙니다.

프로젝트 A와 B가 서로 다른 버전의 같은 라이브러리를 쓸 때 각 환경을 따로 준비할 수 있습니다.
